

ESCUELA COLOMBIANA DE INGENIERÍA JULIO GARAVITO
DEPARTAMENTO DE INGENIERÍA DE SISTEMAS

TechDebtRadar

Proyecto de curso PTIA — Hito 2

Grupo [por confirmar con el docente]

Integrantes:
David Alejandro Contreras Delgado, Andrés Sabogal

Fecha de entrega: 12 de octubre de 2026

Declaración firmada

"Declaro que he escrito este trabajo de investigación por mí mismo, y que no he utilizado otras fuentes o recursos que los indicados para su preparación. Declaro que he indicado claramente todas las citas directas e indirectas, y que este documento no se ha presentado en otro lugar para fines de examen o publicación"

Nombre del estudiante & Firma:

David Alejandro Contreras Delgado

Fecha: 12 de octubre de 2026

Andrés Sabogal

Fecha: 12 de octubre de 2026

Índice de contenido

1. Introducción **4**
1.1. Tres problemas de interés . . . 4
1.2. ¿Por qué como proyecto de PTIA? . . . 4

2. Trabajos relacionados **6**
2.1. Deserción estudiantil . . . 6
2.2. Deuda técnica y calidad de código . . . 6
2.3. Triage de tickets de soporte técnico . . . 7

3. Descripción del problema **8**
3.1. Criterios de selección . . . 8
3.2. Problema seleccionado . . . 8

4. Justificación **9**
4.1. ¿Cuál es el gran problema que se quiere resolver? . . . 9
4.2. ¿Por qué es importante resolverlo? . . . 9

5. Alcance y Objetivos **10**
5.1. Problema específico a resolver . . . 10
5.2. ¿Por qué es una simplificación adecuada? . . . 10
5.3. Objetivo general . . . 11
5.4. Objetivos específicos y criterios de éxito . . . 11
5.5. Delimitación del alcance . . . 12

6. Diseño metodológico **13**
6.1. Estrategia general . . . 13
6.2. ¿Dónde se concentra la inteligencia de la solución? . . . 13
6.3. Herramienta seleccionada y justificación . . . 14
6.4. Arquitectura: componentes y relaciones . . . 15
6.5. Componente principal: LLM y motor de priorización . . . 17
6.6. Propuesta de fórmula de priorización . . . 18
6.7. Prototipo: qué es navegable, qué es simulado y qué está implementado . . . 19

7. Análisis de resultados **20**

8. Conclusiones **21**

Índice de Tablas

1. Objetivos específicos y sus criterios de éxito . . . 11
2. Qué entra y qué no entra en el alcance del proyecto . . . 12
3. Herramientas seleccionadas por componente . . . 14
4. Responsabilidades, entradas y salidas por componente . . . 16
5. Variables de la fórmula de priorización . . . 18

Índice de figuras

1. Arquitectura propuesta de TechDebtRadar: componentes y relaciones. El recuadro verde indica lo implementado y navegable en el Hito 2; el recuadro rojo indica lo diseñado para el Hito 3. 15
2. Flujo de datos de TechDebtRadar: de la URL del repositorio a la recomendación priorizada. 17

1. Introducción

Como equipo evaluamos varias problemáticas del entorno cotidiano de un estudiante de últimos semestres de Ingeniería de Sistemas antes de decidirnos por la que se desarrolla en este documento. A continuación presentamos los tres problemas que más nos interesaría resolver.

1.1. Tres problemas de interés

1. **Detección temprana de riesgo de deserción estudiantil.** Usar datos académicos y socioeconómicos históricos de una institución de educación superior para identificar, desde los primeros semestres, a los estudiantes con mayor probabilidad de abandonar sus estudios, de forma que bienestar universitario pueda intervenir a tiempo.
2. **Detección y priorización automática de deuda técnica en proyectos de software.** Apoyar a equipos de desarrollo pequeños (como el nuestro) a identificar, explicar en lenguaje natural y priorizar qué partes de un código base conviene refactorizar primero, usando modelos de lenguaje de gran escala (LLM) como complemento –no reemplazo– de las reglas estáticas tradicionales.
3. **Triage inteligente de tickets de soporte técnico.** Clasificar y priorizar automáticamente las solicitudes que llegan a una mesa de ayuda (por ejemplo, la de la Dirección de Tecnologías de Información de la universidad), reduciendo el tiempo de respuesta a incidentes críticos y descargando de trabajo repetitivo al personal de soporte de primer nivel.

1.2. ¿Por qué como proyecto de PTIA?

Los tres problemas comparten características que los hacen apropiados para un proyecto del curso de Proyecto en Tecnologías de Inteligencia Artificial:

- Ninguno se resuelve con una simple automatización determinística: los tres requieren **aprender de datos históricos o de conocimiento experto** para apoyar una decisión que finalmente sigue siendo humana (un asesor de bienestar, un desarrollador, un agente de soporte).
- Los tres permiten combinar **razonamiento simbólico o reglas de negocio con aprendizaje estadístico o modelos de lenguaje**, que es precisamente el tipo de "inteligencia híbrida" (conocimiento + razonamiento) que el curso busca que exploremos.
- Los tres tienen **relevancia real y trabajo previo documentado** tanto en la academia como en la industria (ver Sección 2), lo que nos da una línea base cuantitativa contra la cual comparar nuestros propios resultados.
- Los tres son **abordables por un equipo de dos personas en un semestre**: existen conjuntos de datos abiertos o generables, y ninguno exige infraestructura de cómputo costosa, sino el uso de modelos de lenguaje ya entrenados (a través de APIs o modelos open source) y técnicas de *prompting* vistas en el curso.

En la Sección 3 explicamos con qué criterios elegimos, de estos tres, el problema que efectivamente desarrollaremos durante el semestre.

2. Trabajos relacionados

Para cada uno de los tres problemas presentados en la introducción, identificamos dos trabajos relacionados: uno de corte más académico y uno más cercano a una herramienta o sistema en producción, describiendo en ambos casos la tecnología usada y los resultados obtenidos.

2.1. Deserción estudiantil

Amaya Torrado et al. (Amaya Torrado et al., 2014). Construyeron un modelo predictivo de deserción para la Universidad Francisco de Paula Santander (Ocaña) usando árboles de decisión (algoritmos ID3 y C4.5, implementado como J48 en la herramienta WEKA) sobre 201 registros de estudiantes con 16 atributos académicos, demográficos y socioeconómicos recolectados a lo largo de seis años. Al validar el modelo J48 contra los 14 estudiantes que efectivamente desertaron en 2012, el sistema identificó correctamente a 9 de ellos, con 12 falsos positivos adicionales; los autores concluyen que ID3 logra mayor precisión que J48 al generar más reglas (22 frente a 6).

SPADIES (Ministerio de Educación Nacional de Colombia, 2024). El Sistema de Prevención y Análisis de la Deserción en las Instituciones de Educación Superior, operado por el Ministerio de Educación Nacional de Colombia, es la herramienta oficial que centraliza a nivel nacional el seguimiento de la deserción usando modelos estadísticos sobre variables académicas, socioeconómicas e institucionales reportadas por cada universidad. Es un ejemplo de un sistema ya en producción, pero que depende de datos institucionales consolidados a los que un proyecto de curso no tendría acceso directo.

2.2. Deuda técnica y calidad de código

Potdar y Shihab (Potdar & Shihab, 2014). Este es el trabajo seminal sobre *Self-Admitted Technical Debt* (SATD): deuda técnica que el propio desarrollador reconoce en comentarios del código (por ejemplo, `// TODO: arreglar esto luego`). Analizando los comentarios de cuatro proyectos open source de gran tamaño (Eclipse, Chromium OS, Apache HTTP Server y ArgoUML) mediante patrones de lenguaje natural, encontraron que la SATD está presente en entre el 2.4 % y el 31 % de los archivos de un proyecto, y que solo entre el 26.3 % y el 63.5 % de la deuda admitida llega a eliminarse posteriormente. La tecnología usada fue minería de texto sobre comentarios de código, sin modelos de lenguaje modernos.

Souza et al. (Souza et al., 2026). En un trabajo mucho más reciente, evaluaron cuatro LLM (DeepSeek-R1, GPT-5 mini, Llama-3.3 y Qwen2.5-Coder) en la tarea de detectar *code smells*, usando *chain-of-thought prompting* (Wei et al., 2022) contra un conjunto de 268 candidatos etiquetados manualmente por 76 desarrolladores en 30 proyectos Java. Para smells estructurales como *Large Class* lograron un F1-score de hasta 0.93 combinando los cuatro modelos con herramientas de análisis estático mediante votación; sin embargo, para smells más subjetivos como *Refused Bequest* ningún modelo superó un F1 de 0.40. Este trabajo es la referencia tecnológica más cercana a nuestra propuesta, pues confirma que los LLM son competitivos para deuda técnica "objetiva" pero aún tienen margen de mejora en los casos que requieren juicio de diseño.

2.3. Triage de tickets de soporte técnico

Benitez Pereira et al. (Benitez Pereira et al., 2023). Entrenaron una red neuronal totalmente conectada (500 neuronas de entrada correspondientes a las palabras más frecuentes, 4 capas ocultas con activación ReLU y *dropout* del 30 %, y una capa de salida softmax de 7 categorías) usando TensorFlow sobre 2,229 tickets de soporte multilingües. El modelo alcanzó un 75 % de precisión promedio entre las 7 categorías (con un máximo de 100 % en "End of Life" y un mínimo de 41 % en "Active Directory"), y los autores desplegaron un prototipo web funcional con FastAPI, VueJS y PostgreSQL sobre AWS.

Kanimozhi Selvi et al. (Kanimozhi Selvi et al., 2025). Compararon modelos clásicos (Regresión Logística, SVM, Random Forest, XGBoost) y modelos profundos (BERT, RNN, CNN) para categorizar y priorizar 78,313 quejas de clientes de una institución financiera, usando TF-IDF y factorización de matrices no negativas (NMF) para la categorización por departamento, y detección de urgencia basada en NLP para la prioridad. La Regresión Logística obtuvo la mejor exactitud en categorización (91.99 %, frente a 81 % de BERT), mientras que los modelos basados en árboles (Decision Tree y XGBoost) alcanzaron hasta 99.96 % de exactitud en la clasificación de urgencia, superando también a BERT (92.64 %).

3. Descripción del problema

3.1. Criterios de selección

Para escoger, de los tres problemas de la introducción, aquel que desarrollaríamos durante el semestre, definimos los siguientes criterios:

- **Disponibilidad de datos sin restricciones de privacidad.** En Colombia, los datos académicos y socioeconómicos de estudiantes están protegidos por la Ley 1581 de 2012 (habeas data), por lo que un proyecto de curso difícilmente podría acceder a ellos a tiempo. El código fuente público en GitHub bajo licencias abiertas, en cambio, no tiene esa restricción.
- **Viabilidad técnica y de tiempo** para un equipo de dos personas dentro de las 16 semanas del curso, sin depender de infraestructura de cómputo costosa.
- **Relevancia directa con nuestro perfil profesional** como futuros ingenieros de sistemas, y oportunidad de aplicar las técnicas de LLM y *prompting* vistas en el curso.
- **Existencia de trabajo relacionado reciente** que sirva de línea base cuantitativa (F1-score, precisión) contra la cual comparar nuestros propios resultados.
- **Potencial de crecimiento incremental**: que el problema permita evolucionar de un PoC simple (Hito 1-2) hacia un prototipo funcional tipo MVP (Hito 3) sin cambiar de enfoque a mitad de camino.
- **Riesgo ético manejable**: preferimos un problema que no involucre datos personales sensibles ni decisiones que afecten directamente el bienestar de terceros vulnerables sin supervisión humana.

3.2. Problema seleccionado

Aplicando estos criterios, descartamos la **deserción estudiantil** porque exige datos institucionales sensibles a los que no tendríamos acceso oportuno, y el **triage de tickets de soporte** porque requeriría un convenio con una mesa de ayuda real y sus datos históricos, algo que no podíamos garantizar para el inicio del curso.

Seleccionamos entonces el segundo problema: **la detección y priorización de deuda técnica en proyectos de software mediante modelos de lenguaje**. Es un problema que podemos estudiar íntegramente sobre repositorios públicos de código, para el cual ya contamos con experiencia previa como desarrolladores para evaluar cualitativamente si las explicaciones generadas por un LLM tienen sentido, y sobre el cual existe trabajo previo reciente con el que podemos comparar cuantitativamente nuestros resultados de F1-score (Souza et al., 2026).

4. Justificación

4.1. ¿Cuál es el gran problema que se quiere resolver?

El problema de fondo es que la **deuda técnica** –código que funciona pero que fue construido de forma apresurada, duplicada, sin pruebas o difícil de entender– se acumula silenciosamente en casi cualquier proyecto de software y, cuando no se gestiona a tiempo, termina frenando la capacidad de un equipo para seguir agregando valor: cada nueva funcionalidad tarda más en implementarse, aparecen más errores y el código se vuelve cada vez más difícil de mantener para quien no lo escribió originalmente.

Las herramientas tradicionales de análisis estático (por ejemplo SonarQube (SonarSource, 2026)) ayudan a detectar deuda técnica, pero tienden a generar una gran cantidad de alertas de bajo valor, sin explicar en lenguaje entendible por qué un hallazgo importa ni en qué orden conviene atenderlos. Nuestra propuesta es explorar si un modelo de lenguaje, combinado con métricas de análisis estático, puede no solo detectar estos problemas sino también **explicarlos y priorizarlos** de una forma más cercana a como lo haría un desarrollador senior revisando el código de un compañero.

4.2. ¿Por qué es importante resolverlo?

El costo de no gestionar la deuda técnica no es un problema menor ni exclusivamente académico. El reporte de 2022 del *Consortium for Information and Software Quality* (CISQ) estimó que los problemas de baja calidad de software le costaron a la economía de Estados Unidos cerca de USD 2.41 billones en un solo año, de los cuales aproximadamente USD 1.52 billones corresponden directamente al costo acumulado de deuda técnica (CISQ - Consortium for Information and Software Quality, 2022). A esa escala, es evidente que el problema afecta tanto a grandes empresas como a equipos pequeños.

Y son justamente los equipos pequeños –startups, proyectos académicos como el nuestro, o pymes de desarrollo de software, que representan una parte importante del sector TI en Colombia– los que menos recursos tienen para dedicar un rol exclusivo a la calidad del código, y quienes más se beneficiarían de una herramienta liviana que les diga, en lenguaje claro, qué archivos o funciones conviene refactorizar primero y por qué. Adicionalmente, para nosotros como estudiantes de últimos semestres de Ingeniería de Sistemas, entender y gestionar la deuda técnica de un proyecto es una habilidad profesional que necesitaremos en cualquier trabajo de desarrollo de software que emprendamos después de graduarnos, lo que hace de este problema una elección tanto pertinente para el curso como útil para nuestra propia formación.

5. Alcance y Objetivos

5.1. Problema específico a resolver

De la problemática general descrita en la Sección 4 –la deuda técnica se acumula y nadie la explica ni la prioriza en lenguaje claro– en este hito acotamos un problema concreto y medible: **dado un repositorio público de GitHub escrito en Python o JavaScript, identificar automáticamente los archivos con mayores señales de deuda técnica, explicar en lenguaje natural por qué cada uno importa, y producir una lista ordenada por prioridad de refactorización**, combinando métricas de análisis estático con el criterio de un modelo de lenguaje.

No buscamos resolver la deuda técnica (es decir, no refactorizamos código automáticamente), sino **apoyar la decisión** de qué atender primero, de la misma forma en que un desarrollador senior orientaría a uno junior al revisar un código base desconocido.

5.2. ¿Por qué es una simplificación adecuada?

Acotar el problema de esta manera es razonable por cuatro motivos:

- **Cobertura representativa con esfuerzo acotado.** Las cuatro señales de deuda técnica que elegimos cubrir (complejidad ciclomática alta, comentarios de deuda auto-admitida o SATD, duplicación de código y funciones/archivos excesivamente largos) son, según la literatura revisada en la Sección 2, de las más estudiadas y con mejor soporte de herramientas (McCabe, 1976; Potdar & Shihab, 2014), sin pretender cubrir la totalidad de tipos de deuda técnica (por ejemplo, deuda arquitectónica o de pruebas, que quedan fuera de alcance).
- **Dos lenguajes, no todos.** Limitarnos a Python y JavaScript nos permite usar herramientas de análisis estático maduras y de código abierto para ambos (ver Sección 6), evitando construir analizadores propios para cada lenguaje, mientras seguimos cubriendo dos de los lenguajes más usados en proyectos públicos de GitHub.
- **Control del costo y el contexto del LLM.** Enviar un repositorio completo a un modelo de lenguaje es costoso y poco preciso. Al usar las métricas estáticas para preseleccionar solo los archivos más sospechosos (ver *selección de fragmentos relevantes*, Sección 6), acotamos cuánto código debe interpretar el LLM, lo que es viable con el presupuesto de un proyecto de curso.
- **Es un PoC explicable, no una herramienta de producción.** Para un equipo de dos personas en un semestre, es más valioso demostrar que la combinación métricas+LLM produce explicaciones y una priorización razonables en un conjunto acotado de repositorios de prueba, que intentar cubrir todos los lenguajes y tipos de deuda desde el Hito 2.

5.3. Objetivo general

Desarrollar un asistente basado en un modelo de lenguaje que, a partir de la URL de un repositorio público de GitHub en Python o JavaScript, detecte, explique en lenguaje natural y priorice los archivos con mayores señales de deuda técnica, de forma que un equipo de desarrollo pequeño pueda decidir en minutos qué refactorizar primero.

5.4. Objetivos específicos y criterios de éxito

Tabla 1
Objetivos específicos y sus criterios de éxito

Objetivo específico	**Tipo**	**Criterio de éxito (a medir en el Hito 3)**
Calcular métricas estáticas de complejidad, SATD, duplicación y tamaño por archivo.	Cuantitativo	Las métricas se calculan sin error para el 100 % de los archivos soportados (Python/JavaScript) de al menos 5 repositorios de prueba.
Generar explicaciones en lenguaje natural de cada hallazgo.	Cualitativo	En una muestra de 20 hallazgos, al menos el 70 % son calificados como "claros y relevantes" por los dos integrantes del equipo (evaluación ciega, escala Likert 1-5, promedio ≥ 4).
Priorizar los hallazgos combinando métricas y criterio del LLM.	Cuantitativo	El orden de prioridad generado coincide con el orden que el equipo asignaría manualmente en al menos el 70 % de los pares comparados, para una muestra de 20 archivos de 3 repositorios conocidos (métrica de concordancia: correlación de rangos de Spearman ≥ 0.7).
Entregar un prototipo navegable de las 3 pantallas del flujo.	Cualitativo	Un usuario que no participó en el desarrollo completa el flujo analizar $\rightarrow$ priorizar $\rightarrow$ entender sin instrucciones adicionales, en una prueba de usabilidad informal.

Nota importante: estos números son metas que el equipo se propone alcanzar y medir en el Hito 3, no resultados ya obtenidos. En este Hito 2 todavía no existe una implementación funcional del análisis ni del componente LLM –ver la distinción explícita entre lo navegable y lo simulado en la Sección 6.7–, por lo que ningún porcentaje de precisión se reporta todavía como logrado.

5.5. Delimitación del alcance

Tabla 2
Qué entra y qué no entra en el alcance del proyecto

Entradas	URL de un repositorio **público** de GitHub. El sistema analiza únicamente los archivos `.py` y `.js`/`.ts` de la rama principal (`main`/`master`) en el momento del análisis.
Tipos de deuda técnica cubiertos	Complejidad ciclomática alta (McCabe), comentarios de deuda auto-admitida (SATD: `TODO`, `FIXME`, `HACK`, `XXX`), duplicación de código entre archivos, y funciones o archivos excesivamente largos.
Salidas	Lista de archivos ordenada por un *score* de prioridad (Sección 6), con una explicación en lenguaje natural y una sugerencia de refactorización por hallazgo.
Fuera de alcance	Repositorios privados; análisis de *pull requests* o de más de una rama; lenguajes distintos a Python/JavaScript; vulnerabilidades de seguridad (SAST de seguridad, un problema distinto al de mantenibilidad); aplicar refactors automáticamente; y, en este Hito 2, cualquier integración real y funcional con GitHub o con un LLM –el prototipo de interfaz usa datos simulados, ver Sección 6.7.

6. Diseño metodológico

6.1. Estrategia general

La estrategia seleccionada es **híbrida**: combinar reglas deterministas y métricas de análisis estático (que son baratas, reproducibles y ya validadas por la literatura) con un modelo de lenguaje (que aporta lo que las métricas no pueden dar por sí solas: una explicación en lenguaje natural y un juicio cualitativo sobre la severidad real de cada hallazgo en su contexto).

Elegimos esta combinación y no cada extremo por separado por las siguientes razones:

- **Solo métricas estáticas** (el enfoque de herramientas como SonarQube (SonarSource, 2026)) es reproducible y barato, pero –como se argumentó en la Sección 4– produce listados de alertas sin explicación accesible ni priorización orientada a la toma de decisiones de un equipo pequeño.
- **Solo LLM** (pegar el repositorio completo y pedirle que ”encuentre la deuda técnica”) es costoso en tokens, poco reproducible –el mismo repositorio puede recibir evaluaciones distintas en cada corrida– y propenso a que el modelo alucine problemas que no están respaldados por ninguna señal objetiva en el código.
- **La combinación** usa las métricas deterministas para decidir *qué* mirar (reduciendo drásticamente cuánto código debe leer el LLM) y usa el LLM para decidir *cómo explicarlo y qué tan grave es en contexto*, apoyándose en cada herramienta para lo que mejor hace.

6.2. ¿Dónde se concentra la inteligencia de la solución?

Distinguimos dos tipos de ”inteligencia” en el sistema, siguiendo la misma separación que pide el enunciado de este hito:

- **Conocimiento**, que proviene de dos fuentes: (1) las métricas de análisis estático, cuyo ”conocimiento” está codificado como reglas deterministas bien establecidas en la literatura de ingeniería de software (p. ej. los umbrales de complejidad ciclomática de McCabe (McCabe, 1976), o los patrones léxicos de SATD de Potdar y Shihab (Potdar & Shihab, 2014)); y (2) el conocimiento general sobre buenas prácticas de programación que el LLM adquirió durante su preentrenamiento, el cual no re-entrenamos ni ajustamos (ver justificación de no usar *fine-tuning* más abajo).
- **Razonamiento**, que ocurre en dos puntos distintos del flujo: dentro del LLM, al generar una explicación específica para un fragmento de código concreto y al estimar una severidad en contexto (por qué *este* archivo, con *estas* métricas, es más o menos urgente que otro); y en el **motor de priorización determinista** (Sección 6.6), que combina explícitamente las métricas con el juicio del LLM mediante una fórmula fija y auditable, en lugar de delegarle al LLM la decisión final de orden.

Es decir: el LLM no decide solo quién va primero en la lista; su salida (una severidad y una explicación) es

un insumo más, con un peso definido, dentro de un cálculo determinista y reproducible. Esto es intencional: nos interesa que la priorización final se pueda explicar y auditar, no solo justificar diciendo "porque el modelo lo dijo".

6.3. Herramienta seleccionada y justificación

Tabla 3
Herramientas seleccionadas por componente

Componente	Herramienta propuesta	Por qué
Análisis estático (Python)	`radon` (complejidad ciclomática, longitud) y detección de duplicados por *hashing* de bloques	Librería de código abierto, mantenida, con salida programática (JSON) fácil de integrar; evita escribir un analizador de AST propio en el tiempo de un semestre.
Análisis estático (JavaScript)	`lizard` (métricas de complejidad multi-lenguaje)	Misma lógica que `radon` pero con soporte nativo de JavaScript/TypeScript, lo que nos permite reusar un solo flujo de métricas para los dos lenguajes del alcance.
Componente LLM	API de un modelo de lenguaje de propósito general, con *prompting* estructurado (código + métricas + contexto) y encadenamiento de razonamiento (*chain-of-thought*) (Wei et al., 2022)	Evita el costo y el riesgo de sobreajuste de entrenar o ajustar (*fine-tune*) un modelo propio con un conjunto de datos etiquetado que no tenemos; un trabajo reciente que comparamos como línea base (Souza et al., 2026) reporta resultados competitivos usando LLMs de propósito general sobre esta misma tarea, sin necesidad de ajuste fino.
Interfaz del prototipo	HTML, CSS y JavaScript estático (sin *framework*)	Suficiente para navegar las 3 pantallas con datos simulados en este hito; no requiere backend ni proceso de construcción (*build*), lo que simplifica la entrega y ejecución local (ver Sección 6.7).

Por qué no ajustamos (*fine-tuning*) un modelo propio: ajustar un LLM requiere un conjunto de datos etiquetado de tamaño considerable (código con severidad de deuda técnica ya evaluada por expertos), que no existe públicamente para esta tarea en la escala que necesitaríamos y que no podríamos construir de forma confiable en el tiempo del curso. Usar *prompting* estructurado sobre un modelo ya entrenado es la alternativa viable dado nuestro presupuesto de tiempo y de cómputo, y es consistente con

el enfoque de trabajos recientes sobre el mismo problema (Souza et al., 2026).

6.4. Arquitectura: componentes y relaciones

La Imagen 1 muestra los componentes propuestos y cómo se relacionan. Distinguimos explícitamente, mediante los dos recuadros punteados, qué parte está **implementada y es navegable en este Hito 2** (solo la interfaz de usuario, con datos simulados) y qué parte está **diseñada para el Hito 3** (todo el procesamiento real: validación, obtención de código, análisis estático, selección, LLM y priorización), que en este hito no está construida ni probada.

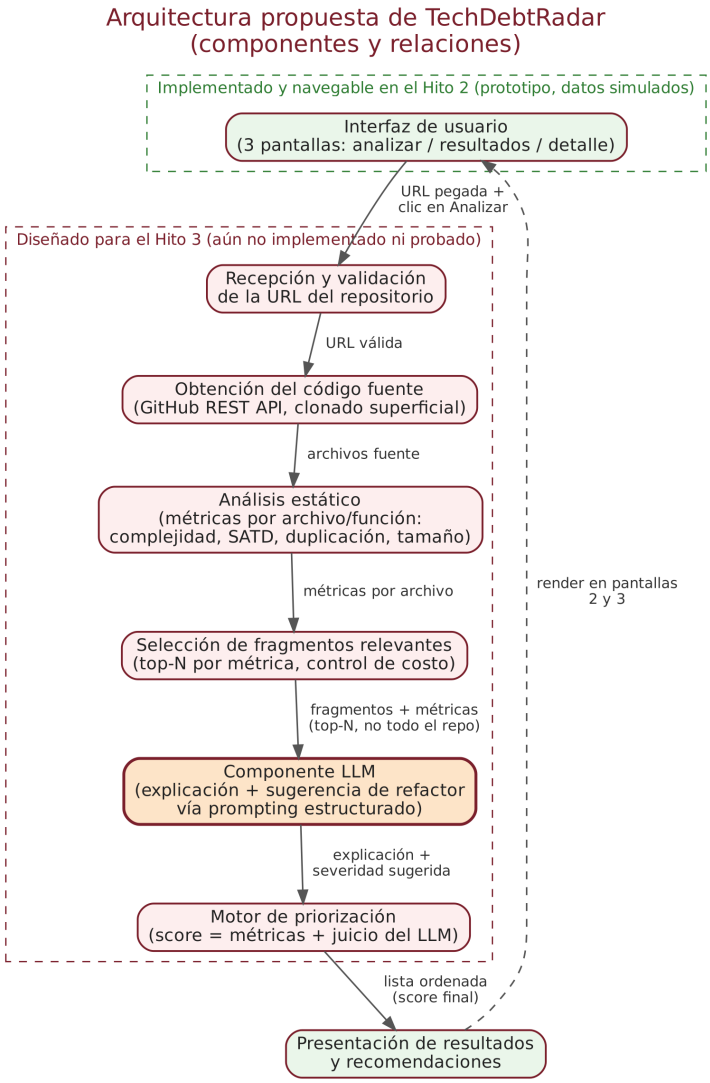


Imagen 1
Arquitectura propuesta de TechDebtRadar: componentes y relaciones. El recuadro verde indica lo implementado y navegable en el Hito 2; el recuadro rojo indica lo diseñado para el Hito 3.

Tabla 4
Responsabilidades, entradas y salidas por componente

Componente	**Responsabilidad**	**Entrada → Salida**
Interfaz de usuario	Recibir la URL, mostrar la lista priorizada y el detalle de cada hallazgo	Interacción del usuario → URL / clics
Recepción y validación	Verificar formato de la URL y que el repositorio sea público y accesible, con manejo explícito de error si no lo es	URL → URL validada o mensaje de error
Obtención del código	Clonado superficial (*shallow clone*) o uso de la API REST de GitHub, limitando tamaño y rama (`main`/`master`)	URL validada → archivos fuente locales
Análisis estático	Calcular complejidad, SATD, duplicación y tamaño por archivo, filtrando por extensión soportada	Archivos fuente → métricas por archivo
Selección de fragmentos	Normalizar métricas y elegir el top-N de archivos más sospechosos, acotando cuánto código se envía al LLM	Métricas por archivo → subconjunto top-N
Componente LLM	Generar explicación en lenguaje natural, severidad (1-5) y sugerencia de refactor por archivo seleccionado, vía *prompting* estructurado	Código + métricas del top-N → explicación + severidad + sugerencia
Motor de priorización	Calcular el *score* final por archivo combinando métricas normalizadas y severidad del LLM (fórmula en 6.6)	Métricas + severidad LLM → lista ordenada
Presentación de resultados	Transformar la lista ordenada y los detalles en las pantallas 2 y 3	Lista ordenada → render en la interfaz

La Imagen 2 detalla el flujo de datos paso a paso, desde que el usuario pega la URL hasta que llega a la pantalla de detalle de un hallazgo, incluyendo los puntos donde se filtra, normaliza y selecciona información para controlar el costo de la llamada al LLM.

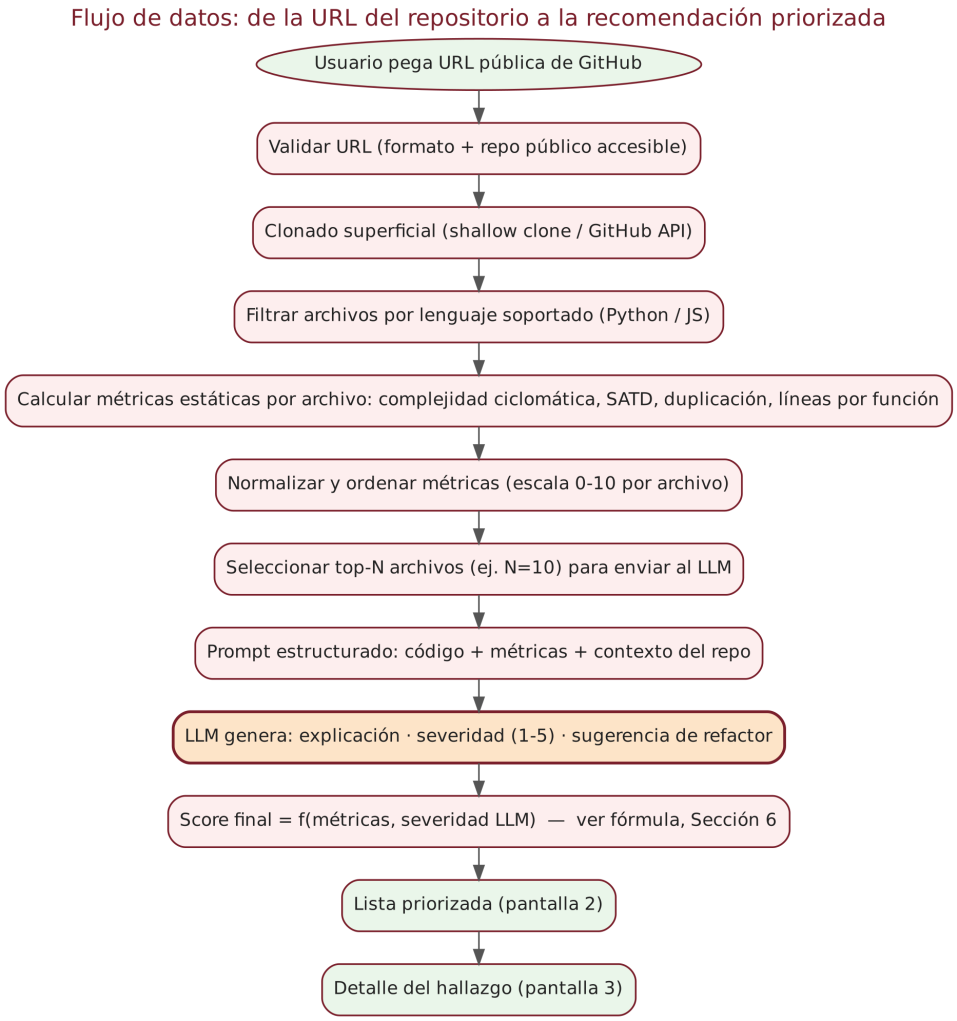


Imagen 2
Flujo de datos de TechDebtRadar: de la URL del repositorio a la recomendación priorizada.

Medidas de control consideradas para el Hito 3 (no implementadas todavía, pero incorporadas en el diseño para no tener que rediseñar la arquitectura más adelante): límite de tamaño de repositorio y de número de archivos a clonar; límite de tiempo (*timeout*) para la clonación y para cada llamada al LLM; manejo explícito de URLs inválidas, repositorios privados o inexistentes, y de archivos binarios o generados (p. ej. carpetas `node_modules`, `dist`, `build`) que deben excluirse del análisis; y saneamiento del contenido de archivos antes de incluirlo en el *prompt*, dado que el código de un repositorio de terceros es, en este contexto, una entrada no confiable.

6.5. Componente principal: LLM y motor de priorización

Consideramos que el **componente LLM** y el **motor de priorización** son, juntos, el componente principal de la solución, porque es donde se concreta la propuesta de valor del proyecto (explicar y priorizar, no solo detectar).

El **componente LLM** recibe, para cada archivo del top-N seleccionado: el código del archivo (o de sus funciones más largas, si el archivo completo excede un límite de tokens razonable), sus métricas ya calculadas (complejidad, cantidad de SATD, porcentaje de duplicación) y un breve contexto del repositorio (nombre, lenguaje, propósito si está disponible en el `README`). Con esta información, el *prompt* estructurado le pide tres salidas separadas: (1) una explicación en lenguaje natural de por qué el archivo representa deuda técnica, (2) una severidad en una escala de 1 a 5, y (3) una sugerencia concreta de refactorización. Usamos *prompting* con encadenamiento de razonamiento (*chain-of-thought*) (Wei et al., 2022), pidiéndole al modelo que primero identifique las señales relevantes del código y luego concluya la severidad, en lugar de pedir la severidad directamente, para reducir la variabilidad de la respuesta.

El **motor de priorización** toma la severidad devuelta por el LLM junto con las métricas normalizadas y calcula un *score* final por archivo, descrito en la Sección 6.6. Separar este cálculo del LLM (en vez de pedirle directamente al modelo que ordene los archivos) es una decisión deliberada: un cálculo determinista es reproducible y auditable –podemos explicar exactamente por qué un archivo quedó primero–, mientras que pedirle a un LLM que ordene una lista completa es menos estable entre corridas y más difícil de justificar ante el usuario.

6.6. Propuesta de fórmula de priorización

Proponemos la siguiente fórmula para calcular el *score* de priorización de un archivo:

$$\text{Score}(\text{archivo}) = w_1 \cdot \text{norm}(CC) + w_2 \cdot \text{norm}(SATD) + w_3 \cdot \text{norm}(DUP) + w_4 \cdot Sev_{LLM}$$

Tabla 5
Variables de la fórmula de priorización

Variable	Significado	Escala / normalización
CC	Complejidad ciclomática promedio del archivo (McCabe) (McCabe, 1976)	Normalizada min-max a $[0, 10]$ sobre los archivos del repositorio analizado
$SATD$	Número de comentarios de deuda auto-admitida (`TODO`/`FIXME`/`HACK`/`XXX`) (Potdar & Shihab, 2014)	Normalizada min-max a $[0, 10]$
DUP	Porcentaje de líneas del archivo que forman parte de un bloque duplicado en otro archivo	Normalizada min-max a $[0, 10]$
Sev_{LLM}	Severidad estimada por el LLM	Escala original 1-5, reescalada linealmente a $[0, 10]$
w_1, w_2, w_3, w_4	Pesos de ponderación de cada señal	$w_1 = 0.25$, $w_2 = 0.15$, $w_3 = 0.20$, $w_4 = 0.40$; $\sum w_i = 1$

La normalización min-max se calcula *dentro de cada repositorio analizado* (no contra una base externa fija), para que el *score* sea relativo al resto de archivos del mismo proyecto y así tenga sentido para equipos

con bases de código de tamaños muy distintos. Le asignamos el peso más alto ($w_4 = 0.40$) a la severidad del LLM porque es la única señal que incorpora contexto semántico del código (qué tan crítico es el archivo dentro de la lógica del proyecto), algo que las métricas puramente sintácticas no pueden capturar por sí solas.

*Advertencia explícita: los pesos $w_1, \ldots, w_4$ y la propia estructura lineal de la fórmula son una **propuesta inicial no validada**, construida a partir de nuestro criterio como desarrolladores y de la literatura revisada, no a partir de un experimento con datos. En el Hito 3 planeamos calibrar estos pesos comparando el orden producido por la fórmula contra el orden que asignaría manualmente nuestro equipo sobre un conjunto de repositorios de prueba (ver el criterio de éxito basado en correlación de Spearman en la Tabla 1), y ajustarlos si la comparación lo justifica. Ningún resultado de esta calibración existe todavía.*

6.7. Prototipo: qué es navegable, qué es simulado y qué está implementado

Para evitar cualquier ambigüedad sobre el estado real del desarrollo, distinguimos explícitamente tres categorías en el prototipo de interfaz entregado con este hito (ver también el documento de prototipo que acompaña esta entrega, y la carpeta `prototype/` del repositorio):

- **Navegación funcional:** las 3 pantallas (ingresar URL, lista priorizada, detalle del hallazgo) existen como páginas reales, enlazadas entre sí, y se pueden recorrer haciendo clic como lo haría un usuario final.
- **Datos simulados:** el contenido que se muestra (nombres de archivos, métricas, severidades, explicaciones y sugerencias de refactor) está **escrito a mano por el equipo** en archivos de datos de ejemplo, para representar visualmente cómo se vería un resultado real. Ninguno de estos datos proviene de analizar un repositorio real.
- **Funcionalidad no implementada todavía:** no existe, en este hito, ningún análisis real de un repositorio de GitHub ni ninguna llamada real a un LLM. Si el usuario pega una URL distinta a la de ejemplo, el prototipo no la analiza; esto se indica explícitamente en la propia interfaz (ver Sección 6.7 del informe de prototipo) para no generar una falsa impresión de funcionalidad.

Esta distinción es la misma que exige el enunciado del hito y se mantiene consistente entre este documento, el prototipo y el guion de video: en ningún entregable se afirma que TechDebtRadar ya analiza repositorios reales o ya usa un LLM en producción.

7. Análisis de resultados

Recomendaciones

Detallar los resultados

- ¿Cuáles son los dos mejores casos de prueba? (Secuencia de entradas y salidas esperadas y logradas)
- ¿Cuál es el caso de prueba más significativo? (Además de la secuencia de entradas y salidas, analicen el resultado)

8. Conclusiones

Recomendaciones

Presentar una reflexión sobre los resultados y hallazgos de su trabajo; lecciones aprendidas

- ¿Cuáles son los dos aciertos y dos errores más relevantes? (Aprendizaje de cada uno de ellos)
- ¿Cuáles son las tres conclusiones más evidentes?
- ¿Cuál es una buena idea de trabajo futuro?

Las conclusiones son una oportunidad para resumir los resultados y hallazgos de una investigación o un proyecto, y para hacer énfasis en la idea o punto principal del escrito al final de su argumento

Referencias

Amaya Torrado, Y. K., Barrientos Avendaño, E., & Heredia Vizcaíno, D. J. (2014). Modelo predictivo de deserción estudiantil utilizando técnicas de minería de datos. https://documentos.redclara.net/bitstream/10786/759/1/124-22-3-2014-Modelo%20predictivo%20de%20desercion%20estudiantil%20utilizando%20tecnicas%20de%20mineria%20de%20datos

Benitez Pereira, L. S., Pizzio, R., Bonho, S., Ferraz de Souza, L. M., & Arnoni Junior, A. C. (2023). Machine Learning for Classification of IT Support Tickets. *IEEE Conference Publication*. https://doi.org/10.1109/document.10051041

CISQ - Consortium for Information and Software Quality. (2022). *The Cost of Poor Software Quality in the US: A 2022 Report* (inf. téc.). CISQ. https://www.it-cisq.org/the-cost-of-poor-software-quality-in-the-us-a-2022-report/

Kanimozhi Selvi, C. S., Jyothi Shri, S., Prasshanthini, R., Neelamegan & Sanjay, R. (2025). Customer Support Ticket Categorization and Prioritization Using NLP. *Proceedings of SCITEPRESS*. https://www.scitepress.org/Papers/2025/136404/136404.pdf

McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering*, *SE-2*(4), 308-320. https://doi.org/10.1109/TSE.1976.233837

Ministerio de Educación Nacional de Colombia. (2024). SPADIES: Sistema de Prevención y Análisis de la Deserción en las Instituciones de Educación Superior. https://www.mineducacion.gov.co/sistemasinfo/spadies/

Potdar, A., & Shihab, E. (2014). An Exploratory Study on Self-Admitted Technical Debt. *IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 91-100. https://doi.org/10.1109/ICSME.2014.31

SonarSource. (2026). SonarQube: Code Quality and Code Security. https://www.sonarsource.com/products/sonarqube/

Souza, S., Santana, A., Figueiredo, E., Muzetti, I., Montandon, J. E., & Briand, L. (2026). Beyond Strict Rules: Assessing the Effectiveness of Large Language Models for Code Smell Detection. *arXiv preprint arXiv:2601.09873*. https://arxiv.org/html/2601.09873v1

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS) 35*. https://arxiv.org/abs/2201.11903